An AI-Powered Multimedia Resource Recommendation System for Education of AI

Student Name: Liqi Luan
Supervisor Name: Yang, Long
Submitted as part of the degree of BSc Computer Science to the
Board of Examiners in the Department of Computer Science, Durham University

Abstract—Recent AI learning products (e.g., NotebookLM and general-purpose LLM assistants) are highly effective at document-grounded question answering and summarisation, but they provide limited support for structured reinforcement learning workflows: they do not explicitly discover external resources across heterogeneous platforms, and they offer weak domain-specific ranking for AI/ML study. This paper presents AI-Pedia, an AI-powered multimedia recommendation system that addresses these gaps. The system contributes: (1) learner-specific topic extraction from uploaded notes using TF-IDF and Maximal Marginal Relevance (MMR); (2) multi-source retrieval from Wikipedia, Google Scholar, arXiv, YouTube, and GitHub; (3) AI-domain filtering and content-based filtering (CBF) ranking to select relevant text, video, and code resources; and (4) optional LLM-generated summaries with a rule-based fallback and a web interface with real-time progress feedback. These contributions improve recommendation relevance, cross-modal resource coverage, and practical usability for self-study, reducing manual search overhead and helping learners map their own materials to high-value reinforcement resources more efficiently. We report the system architecture and implementation, and define an evaluation protocol against NotebookLM and a "dumb student" baseline.

Index Terms—Artificial intelligence, content-based filtering, educational technology, keyword extraction, resource recommendation

1 Introduction

AI education now relies on a dense ecosystem of heterogeneous learning assets: lecture slides, textbooks, research papers, online tutorials, videos, and public code repositories. This breadth of material increases access to knowledge and allows students to choose resources that match their interests. However, it also creates a non-trivial *selection and sequencing* problem: learners must continually decide what to read or watch next, which explanation depth is appropriate, and how to connect abstract theory with executable code examples. Educational and cognitive-load literature indicates that poorly structured information environments can overload working memory and reduce learning efficiency, particularly when learners must split attention across multiple formats and sources without guidance [6]. Multimedia learning research, by contrast, shows that carefully orchestrated combinations of text, audio, video, graphics, animation, and interactivity can substantially improve understanding when aligned with learners' prior knowledge and goals [8]. Fig. 1 illustrates common multimedia modalities that modern AI students encounter.

In parallel with this growth in available resources, recent large language model (LLM) systems have become highly effective at document-grounded explanation and summarisation, enabled by transformer-based architectures and large-scale pre-training [9], [10]. Products such as NotebookLM demonstrate strong conversational support around uploaded material: given a collection of notes, PDFs, or web pages, they can answer questions, produce summaries, and help users navigate the content space. These systems are well suited to on-demand *explanation* of existing resources. However, they are not primarily designed as cross-platform *reinforcement discovery* engines. In practice, current tools rarely (i) optimise explicitly for domain-targeted retrieval from heterogeneous channels (scholarly text, educational video, and code repositories), or (ii) provide transparent, modular control over how newly discovered resources are ranked and filtered with respect to a learner's own corpus.

This project addresses that gap with AI-Pedia: a system that starts from the learner's uploaded documents and produces a curated reinforcement package of external resources. AI-Pedia is intentionally positioned between purely conversational assistants and fully manual web search. The core design choice is to combine established information-retrieval and recommender-system methods, rather than relying on a single monolithic generation step. Specifically, AI-Pedia uses TF-IDF term weighting [1] and diversity-aware

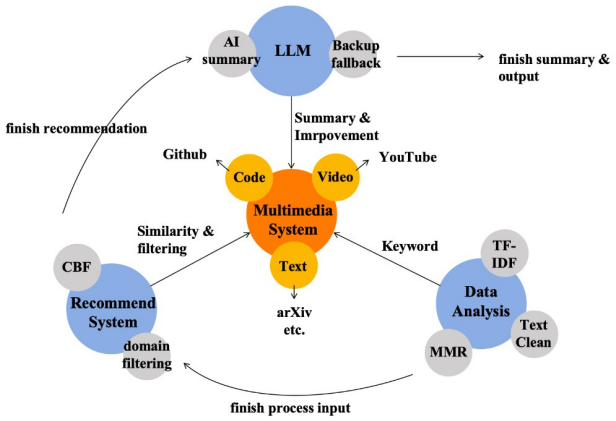

Fig. 1. Multimedia design for learning

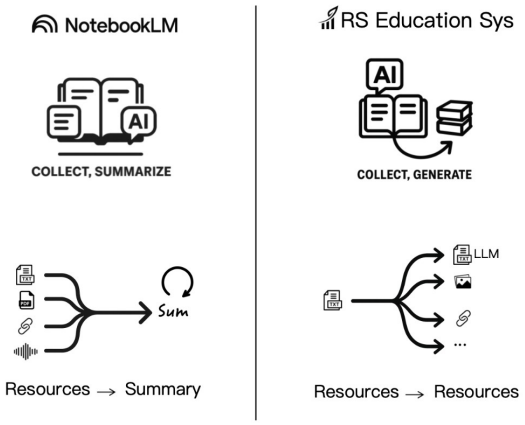

Fig. 2. Contrast between NotebookLM and reinforcement system.

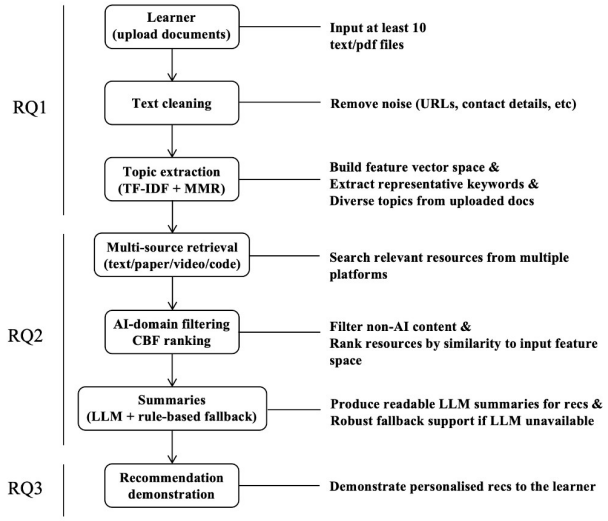

Fig. 3. High-level AI-Pedia pipeline aligned with research questions (RQ1–RQ3).

keyword selection via MMR [2] to extract topic signals from the learner's corpus, performs multi-source retrieval over Wikipedia, Google Scholar, arXiv, YouTube, and GitHub, and then applies content-based filtering (CBF) for personalised ranking [4], [5]. The retrieval and scoring stages follow standard vector-space principles widely used in information retrieval [3]. Fig. 2 contrasts this reinforcement-oriented workflow with the summarisation-oriented workflow of NotebookLM, while Fig. 3 defines the user role, data flow, and rationale behind each design component.

Against this background, the research problem addressed in this paper can be stated as follows: *how can we transform a learner's private AI materials into high-quality external reinforcement recommendations that are relevant, diverse, and operationally usable for self-study?* To give this problem a clear structure, we decompose it into three research questions:

- **RQ1**: can statistically grounded keyword extraction (TF-IDF + MMR) produce stable topic signals from mixed-quality student documents [2]?
- **RQ2**: can multi-source retrieval plus AI-domain filtering improve useful resource coverage while suppressing noisy results from open web search [3]?
- **RQ3**: can CBF ranking and concise summaries improve perceived usefulness, trust, and efficiency compared with a generic interaction-only baseline [7]?

The project objectives follow directly from these questions and translate the research problem into concrete system-building tasks. They are:

- **O1**: design and implement a modular pipeline for topic extraction, multi-source retrieval, filtering, ranking, and packaging;
- **O2**: provide a practical web interface with upload, progress feedback, and one-click result export, so the system is usable in realistic student workflows;
- **O3**: integrate optional LLM summarisation while keeping deterministic fallback summaries for reliability;
- **O4**: define an evaluation protocol that compares AI-Pedia with a strong general-purpose baseline (NotebookLM) and a deliberately weakened "dumb student" baseline to isolate the contribution of each pipeline component.

Beyond these formal objectives, AI-Pedia is designed around concrete student use cases. A typical scenario is a learner preparing for an exam or project who already has a large, messy collection of notes and readings but lacks a clear reinforcement plan: which foundational topics to revisit, what alternative explanations to consult, and how to connect theory with code. Another scenario is a student working on a machine-learning project who needs targeted refreshers on specific subtopics (e.g., regularisation, optimisation, evaluation metrics) and relevant example code. In both cases, manually searching across disparate platforms and judging resource quality is time-consuming and error-prone. Instead of replacing existing materials, AI-Pedia treats them as the anchor for all recommendations, using the pipeline in Fig. 3 to map from the learner's corpus to targeted external resources. This emphasis on "starting from what the student already has" shapes both the algorithmic choices and the user-interface design described later.

Taken together, the contribution of this introduction is to motivate the need for reinforcement-oriented recommendations in AI education, to position AI-Pedia within that problem space, and to justify the key design choices using prior evidence. IR and recommender-system theory support the chosen ranking strategy [3], [4], [5]; learning-science work supports the need for structured, multimodal reinforcement rather than unguided browsing [6], [7]; and modern LLM literature supports integrating model-based summarisation as an assistive layer rather than as the sole mechanism [9], [10]. The remainder of this paper is organised as follows. Section II surveys related work and situates AI-Pedia among educational recommenders, multimedia tools, and LLM-based assistants. Section III describes the methodology and implementation in detail. Section IV presents implementation outcomes. Section V details the evaluation design and planned analyses. Section VI concludes and outlines future directions.

2 Related Work

The design of AI-Pedia is motivated by several strands of prior research, including educational recommender systems, multimedia-based learning environments, and large language model (LLM)-based learning assistants. These areas address different aspects of modern AI education, ranging from resource discovery and ranking to document understanding and interactive explanation. This section situates AI-Pedia within this literature by first outlining the historical evolution of learning support technologies, then reviewing educational recommender systems, multimedia learning tools, and LLM-based assistants, and finally summarising the key similarities and differences that motivate the AI-Pedia pipeline.

2.1 Historical Overview

Recommender systems have been widely studied for more than two decades and are now common in domains such as e-commerce, entertainment, and digital services. Early work focused on suggesting relevant items such as movies or products, typically using collaborative filtering (CF), content-based filtering (CBF), or hybrid approaches [5], [13]. These methods were later adapted to education, where the objective shifted from recommending consumer items to supporting learning through courses, readings, exercises, and explanatory materials.

In parallel, multimedia learning research showed that combining text, visuals, video, and interactive media can improve understanding when designed according to cognitive principles [7]. This work emphasised that well-coordinated multimodal materials can reduce cognitive load and improve retention.

More recently, advances in artificial intelligence and natural language processing have introduced LLM-based systems capable of summarising documents, answering questions, and supporting interactive exploration. These systems represent a further step in the evolution of learning technologies.

Fig. 4 illustrates this trajectory: from classical recommender systems, through multimedia learning tools and LLM-based assistants, towards reinforcement-oriented recommendation systems such as AI-Pedia. Unlike systems that primarily explain existing content, AI-Pedia focuses on a "resources $\rightarrow$ resources" workflow in which learner documents are used to discover new external reinforcement materials.

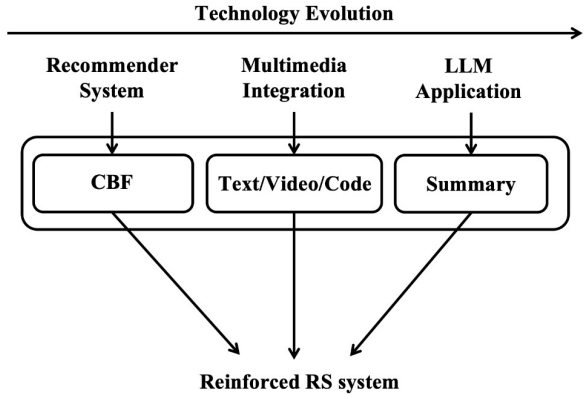

Fig. 4. Technology evolution

2.2 Educational Recommender Systems

Educational recommender systems aim to suggest learning items that match a learner's level, goals, or context. These systems are typically based on content-based filtering, collaborative filtering, and hybrid approaches.

Content-Based Filtering (CBF). CBF methods represent documents using techniques such as TF-IDF and rank resources using similarity measures like cosine similarity. This approach is effective when user data is limited, but is often restricted to closed datasets, limiting dynamic discovery and diversity.

Collaborative Filtering (CF). CF relies on patterns from multiple users. While it improves diversity, it requires large-scale interaction data and suffers from cold-start issues, making it less suitable for personalised, document-driven learning scenarios.

Hybrid Methods. Hybrid systems combine multiple signals but increase system complexity and still depend on behavioural data.

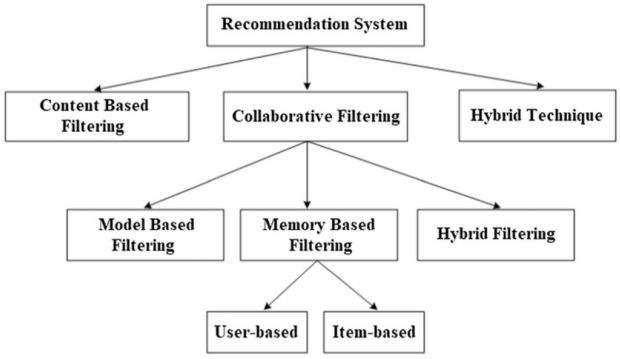

Fig. 5. Taxonomy of recommendation system techniques. Adapted from Shanmugamathan and Almutairi [14].

AI-Pedia adopts a purely content-based strategy and operates over heterogeneous open resources such as Wikipedia, arXiv, YouTube, and GitHub. This enables dynamic discovery but introduces challenges such as noise and heterogeneity, which are addressed through domain filtering and ranking.

2.3 Multimedia-Based Learning Tools

Multimedia learning tools present information across multiple modalities, including text, visuals, and video. Research shows that such multimodal presentations can improve understanding when properly designed [7].

However, most multimedia systems operate within controlled platforms with predefined resources, limiting flexibility and personalisation.

AI-Pedia extends multimedia learning beyond presentation by dynamically retrieving external multimodal resources based on learner documents, using them for reinforcement rather than static delivery.

2.4 LLM-Based Learning Assistants

Recent LLM-based systems enable document-grounded learning through conversational interaction. These systems support summarisation, explanation, and question answering over user-provided materials.

A representative example is NotebookLM, which allows users to organise notes and PDFs and interact with them through a "collect → summarise → interact" workflow. This approach is effective for understanding existing materials and reducing cognitive effort.

However, LLM-based assistants generally do not perform structured discovery of new learning resources. They rely on the uploaded corpus, provide limited transparency in ranking decisions, and may generate broad or uncertain outputs due to the nature of generative models.

This distinction is illustrated in Fig. 6, which compares manual learning, generic LLM assistants, and AI-Pedia. While LLM systems significantly improve efficiency, they still provide limited feedback and lack structured resource discovery. In contrast, AI-Pedia integrates analysis, recommendation, and LLM assistance, enabling stronger feedback, better management, and targeted reinforcement through external multimodal resources.

AI-Pedia therefore complements LLM-based assistants by shifting the focus from explanation to reinforcement-oriented recommendation.

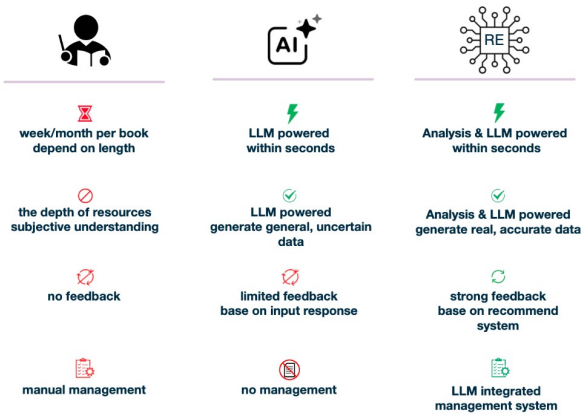

Fig. 6. Qualitative comparison between manual learning, generic LLM assistants, and the AI-Pedia reinforced recommendation system.

2.5 Summary of Commonalities and Differences

Educational recommender systems, multimedia learning tools, and LLM-based assistants all aim to improve learning efficiency. However, they differ in their assumptions and capabilities.

AI-Pedia introduces three key differences: it starts from learner documents rather than fixed datasets, focuses on discovering new reinforcement resources instead of only summarising existing content, and employs an interpretable multi-stage pipeline rather than opaque models.

These characteristics position AI-Pedia as a reinforcement-oriented multimedia recommendation system that bridges recommender systems, information retrieval, and LLM-based assistants.

3 Methodology

This section presents the design and implementation of AI-Pedia in detail. In line with the project objectives, the methodology is organised around a modular pipeline that transforms learner-uploaded study materials into curated external reinforcement resources. The section first defines the overall problem formulation and system architecture, then describes the major functional modules of the system, and finally discusses the engineering implementation of the full prototype.

3.1 Overall System Design and Problem Formulation

The core problem addressed in this project is the following: given a learner's private corpus of AI-related documents, how can a system automatically identify the most important topics, retrieve relevant reinforcement materials from heterogeneous online sources, and rank these materials in a way that is useful for self-study?

This problem was formulated as a pipeline rather than a single end-to-end generation task. The motivation for this design is twofold. First, a modular architecture improves interpretability: each stage of the recommendation process can be analysed, debugged, and evaluated independently. Second, the system is intended to support reinforcement-oriented study rather than merely explanation of existing documents. Therefore, the output must consist of newly discovered external resources, not only summaries of uploaded materials.

At a high level, AI-Pedia takes as input a ZIP archive containing a learner's AI/ML-related TXT or PDF documents. The system then processes this input through five stages:

1) **Document preprocessing and corpus construction**: uploaded files are extracted, validated, and converted into a clean text corpus;
2) **Topic extraction**: the system identifies representative keywords and topic phrases from the learner's corpus using TF-IDF and maximum marginal relevance (MMR);
3) **Multi-source retrieval**: candidate reinforcement resources are retrieved from multiple external platforms, including Wikipedia, Google Scholar, arXiv, YouTube, and GitHub;
4) **Filtering and ranking**: the candidate set is filtered for AI-domain relevance and then ranked using content-based filtering;
5) **Packaging and presentation**: the selected resources are summarised, stored, and presented through a web-based interface with progress feedback and downloadable outputs.

This formulation maps directly to the research questions introduced earlier. Topic extraction addresses RQ1, multi-source retrieval and filtering address RQ2, and ranking plus summary generation address RQ3. The system architecture is therefore designed not only as an implementation choice but also as an experimental decomposition of the overall research problem.

The resulting workflow can be understood as a learner-driven resource reinforcement pipeline: instead of starting

from a fixed course catalogue, the system begins from what the learner already has and builds outward towards external, cross-modal resources. This design distinguishes AI-Pedia from both platform-bound recommenders and document-grounded conversational assistants.

3.2 Topic Extraction from Learner Documents

The first major component of the system is learner-specific topic extraction. Its role is to transform a heterogeneous document collection into a compact representation of the learner's current study focus.

3.2.1 Input preprocessing

The system accepts a ZIP archive containing at least 10 TXT or PDF documents. Uploaded PDF files are converted into TXT format before analysis, and the extracted texts are collected into a single corpus for downstream processing. The implementation also removes obvious noise such as hidden macOS files and performs basic text cleaning including lowercasing, punctuation removal, whitespace normalisation, and lightweight phrase normalisation. This preprocessing is necessary because student-provided documents are often messy, inconsistent, and mixed in quality. The project README and implementation both reflect this document-centred workflow and PDF-to-text conversion process.

3.2.2 TF-IDF-based salience estimation

To identify candidate topic terms, the system uses term frequency-inverse document frequency (TF-IDF), a standard information retrieval technique that estimates the importance of a term relative to the learner's corpus. Intuitively, TF-IDF gives higher weight to words and phrases that occur frequently within the learner's documents but are not uniformly common across all documents.

Let t denote a term and d a document. The TF-IDF score used conceptually in the system can be written as:

$$\mathrm{tfidf}(t, d) = \mathrm{tf}(t, d) \cdot \log \frac{N}{\mathrm{df}(t)} \tag{1}$$

where N is the number of documents in the corpus and $\mathrm{df}(t)$ is the number of documents containing term t.

In implementation terms, the keyword extraction module uses TF-IDF vectorisation over the uploaded document set to identify salient phrases. This design is lightweight, deterministic, and appropriate for the relatively small educational corpora targeted by the system. The keyword extraction code explicitly implements TF-IDF-driven topic identification and text-cleaning logic.

3.2.3 Diversity-aware keyword selection with MMR

A limitation of pure TF-IDF ranking is redundancy: many of the highest-scoring phrases can be near-duplicates or semantically overlapping. To avoid returning several variants of the same concept, AI-Pedia applies maximum marginal relevance (MMR) as a diversity-aware selection mechanism.

Conceptually, MMR balances two objectives:

- selecting phrases that are highly relevant to the corpus, and
- discouraging phrases that are too similar to keywords already selected.

This produces a more diverse topic set and improves downstream retrieval coverage. For example, instead of returning several highly similar phrases related only to "neural network training", MMR can promote a broader keyword mix such as "gradient descent", "regularization", "transformer", and "overfitting" when these are all salient in the corpus.

The role of this stage is therefore not only to find important terms, but to create a stable, compact topic representation suitable for multi-platform search. This directly supports the project's first research question concerning whether statistically grounded keyword extraction can produce useful topic signals from mixed-quality student documents.

3.3 Multi-Source Retrieval and AI-Domain Filtering

Once a set of learner-specific keywords has been extracted, the next task is to retrieve potentially useful external resources. This is the core discovery component of AI-Pedia and one of its major distinctions from systems that only summarise user-provided material.

3.3.1 Retrieval across heterogeneous platforms

AI-Pedia retrieves candidate resources from multiple external sources, chosen to reflect the multimodal nature of AI learning. The current implementation searches:

- **Wikipedia**, for high-level concept explanations;
- **Google Scholar**, for academic and survey-style references;
- **arXiv**, for research papers and preprints;
- **YouTube**, for lecture and tutorial videos;
- **GitHub**, for code repositories and implementation examples.

These sources were selected because they correspond to common reinforcement needs in AI education: conceptual overview, scholarly depth, video explanation, and executable code. This multi-source design is explicitly described in the paper abstract and in the project implementation documents.

The retrieval strategy is source-specific rather than uniform. For example, Wikipedia articles are fetched as text explanations, Google Scholar and arXiv are used for academic resource discovery, YouTube is queried for educational video content, and GitHub is used to locate repositories relevant to the extracted topics. The resource search module contains separate search routines for these channels, including English-content filtering and result deduplication.

3.3.2 Why multi-source retrieval is necessary

A single-source design would be insufficient for AI education because learners often need reinforcement in different forms. A textbook-like explanation may help with intuition, a video may help with procedural understanding, and a code repository may help connect theory to implementation. Therefore, the retrieval stage is designed as a cross-modal candidate generator rather than as a simple web search wrapper.

This design also aligns with the learner scenarios described earlier: students revising for exams need concise

conceptual reinforcement, whereas project-based learners may need both papers and example code. The system therefore treats modality diversity as part of the recommendation problem rather than as an optional add-on.

3.3.3 AI-domain filtering

A major challenge in open retrieval is noise. Searching the web or broad content platforms with generic keywords can return irrelevant or low-value results. To address this, AI-Pedia applies an AI-domain filtering layer before ranking.

The resource search and recommendation modules maintain a curated list of AI-relevant keywords, including core machine learning terms, neural network architectures, NLP/CV terminology, optimisation concepts, and common tooling terms. Candidate resources are examined through their titles, descriptions, URLs, and extracted text, and resources that clearly match irrelevant patterns are filtered out unless they still contain strong AI-related evidence. The recommender implementation also includes explicit checks for obviously irrelevant report-like or policy-like material.

This stage is important because the goal is not simply to maximise recall, but to improve useful coverage while suppressing noise. In other words, the system should return fewer irrelevant results without overly narrowing the learner's reinforcement space.

3.4 Content-Based Ranking and Summary Generation

After candidate resources have been retrieved and filtered, the system must decide which ones are most useful for the learner. AI-Pedia addresses this through content-based ranking followed by optional summary generation.

3.4.1 Content-based filtering for personalised ranking

The ranking stage uses content-based filtering (CBF), a recommendation strategy that compares the content of retrieved resources with the learner's own document corpus. Unlike collaborative filtering, which would require large-scale multi-user behaviour data, CBF is well suited to the project context because the system operates primarily on a single learner's uploaded material.

In implementation, both learner documents and candidate resources are represented using TF-IDF-style vectorisation, and cosine similarity is used to score how closely each retrieved resource matches the learner's content profile. Conceptually, the similarity score can be expressed as:

$$\mathrm{sim}(q, r) = \frac{\mathbf{q} \cdot \mathbf{r}}{\|\mathbf{q}\|\|\mathbf{r}\|} \tag{2}$$

where $\mathbf{q}$ denotes the learner-profile vector and $\mathbf{r}$ the candidate resource vector.

Resources with higher similarity scores are treated as more likely to reinforce the learner's current study topics. The recommender module explicitly implements TF-IDF vectorisation, cosine similarity, and content-based recommendation logic.

3.4.2 Why CBF was chosen

CBF was selected over collaborative filtering for three reasons:

- the project does not assume access to large-scale user interaction histories;
- recommendations must be anchored in the learner's own uploaded documents;
- interpretability is improved because relevance can be explained through content similarity rather than hidden behavioural correlations.

This makes CBF a natural fit for a learner-driven reinforcement system.

3.4.3 Resource summaries

To improve usability, the final recommended resources can be accompanied by short summaries. AI-Pedia supports two modes:

- **Optional LLM-generated summaries**, using an OpenAI-based summarisation module when an API key is available;
- **Rule-based fallback summaries**, ensuring that the system remains functional even without external model access.

This dual design reflects a practical engineering decision. LLM summaries can improve readability and user experience, but the recommendation pipeline itself must remain robust and reproducible even when API access is unavailable or fails. The summarisation module explicitly supports this optional-AI plus deterministic-fallback workflow.

The use of LLMs is therefore intentionally limited to an assistive layer. The core retrieval and ranking pipeline remains interpretable and deterministic, while summary generation enhances presentation quality without becoming a single point of failure.

3.5 System Engineering and Prototype Implementation

The final AI-Pedia prototype is implemented as a web-based engineering system rather than as a standalone algorithmic script. This is important because one of the project objectives is usability in realistic student workflows, not only offline recommendation quality.

3.5.1 Software architecture

The system is implemented in Python using Flask as the main web framework. The project is organised into modular backend and frontend components. The backend contains the core modules for keyword extraction, resource search, recommendation, and summarisation, while the frontend provides pages for upload, help, progress display, and optional AI enhancement. The project structure described in the README shows this modular separation clearly.

At the backend level, the major self-developed modules are:

- Keyword extractor: topic extraction and phrase cleaning;
- Resource searcher: multi-source retrieval and AI-domain filtering;

- Recommender: content-based ranking and recommendation selection;
- `ai_summarizer.py`: optional LLM summary generation with fallback behaviour;
- Web app: Flask routes, upload handling, progress management, and result packaging.

This directly addresses the project requirement to describe not only the conceptual method but also the engineered system and which parts were implemented by the author.

3.5.2 User workflow

From the user's perspective, the system workflow is as follows:

1) prepare a ZIP file containing at least 10 AI/ML-related TXT or PDF documents;
2) upload the ZIP archive through the web interface;
3) allow the system to extract text, identify keywords, retrieve candidate resources, and compute recommendations;
4) inspect the generated reinforcement materials and download the packaged results.

The application provides progress feedback during this process and stores outputs in structured folders before packaging them into a downloadable ZIP file. This behaviour is reflected in both the startup guide and the Flask application implementation.

3.5.3 Engineering contribution

From an engineering perspective, the project contribution is not limited to assembling existing tools. The novelty lies in integrating several independently motivated components into a coherent learner-driven reinforcement system:

- topic extraction tailored to noisy learner corpora;
- cross-platform resource discovery across text, video, and code;
- AI-domain filtering to suppress open-web noise;
- interpretable CBF ranking grounded in the learner's own materials;
- optional LLM summaries combined with deterministic fallback logic;
- a usable web prototype with upload, progress tracking, and export.

Thus, the methodology combines three levels of contribution noted in the project notes: a conceptual contribution (a reinforcement-oriented architecture), an algorithmic contribution (the integration of TF-IDF, MMR, and CBF for learner-driven recommendation), and an engineering contribution (a deployable end-to-end prototype).

Overall, the methodology was designed so that each major system stage can later be evaluated independently. This modularity is particularly valuable for the next two sections: the Results section can describe what the system produces in practice, while the Evaluation section can assess the contribution of topic extraction, retrieval, filtering, and ranking in a controlled manner.

The pipeline is modular; each stage can be configured or extended independently.

TABLE 1
Main pipeline stages of AI-Pedia

Stage	Description
1. Upload	User uploads ZIP of TXT/PDF documents (min. 10).
2. Keyword extraction	TF-IDF + MMR; output list of queries.
3. Multi-source search	Wikipedia, Scholar, arXiv, YouTube, GitHub, etc.
4. CBF ranking	Cosine similarity; threshold and AI-keyword filter.
5. Summarisation	Optional LLM or rule-based fallback.
6. Output	ZIP with `txt/`, `video/`, `code/` folders.

4 RESULTS

This section reports what the implemented prototype produces in practice and summarises the main observable engineering outcomes. Rather than presenting only a conceptual pipeline, this section shows the concrete artefacts produced by the current system: extracted topic phrases, retrieved external resources, ranked recommendations, packaged output folders, and local execution measurements.

4.1 Pilot Corpus and Produced Outputs

To validate the end-to-end workflow, the current prototype was exercised on two focused AI/ML corpora bundled with the project. The first corpus, *foundations_ml*, contains ten documents centred on introductory machine learning concepts such as supervised learning, optimisation, feature engineering, and model evaluation. The second corpus, *nlp_transformers*, contains ten documents centred on modern NLP topics such as tokenization, attention, transformers, language-model pretraining, and retrieval-augmented generation. Across both corpora, the total token count is 3,177 words. Each corpus individually satisfies the system's minimum ten-document upload threshold while remaining coherent enough to represent a realistic learner scenario. The documents were selected deliberately to preserve topical focus inside each corpus rather than mixing unrelated AI directions into one bundle. In other words, the evaluation no longer relies on a single topic-diverse corpus; it now uses two controlled ten-document corpora that better match how the recommender is supposed to infer learning intent from a coherent input set.

In the current focused-corpus setup, each corpus produces ten final topic phrases after TF-IDF weighting, MMR selection, and post-processing for redundancy reduction. The foundations corpus yields phrases such as *machine learning*, *model evaluation*, and *feature engineering*, while the NLP corpus yields phrases such as *natural language processing*, *attention mechanisms*, and *large language models*. Across both corpora, multi-source retrieval returned 365 unique resources after de-duplication, comprising 176 text items, 144 videos, and 45 code repositories or notebooks. After recommendation and balancing, the final exported output contained 30 resources in total, with each corpus contributing a 15-item multimodal recommendation package.

Operationally, the output structure matches the design goals in Section III. The system generates three user-facing

TABLE 2
Focused Multi-Corpus Evaluation Setup and Aggregated Outputs

Metric	Value
Corpus setting	2 corpora (20 documents total)
Total corpus words	3,177
Final extracted keywords	10 per corpus
Raw text resources	176
Raw video resources	144
Raw code resources	45
Final recommended text resources	10
Final recommended video resources	10
Final recommended code resources	10
Total final recommended items	30

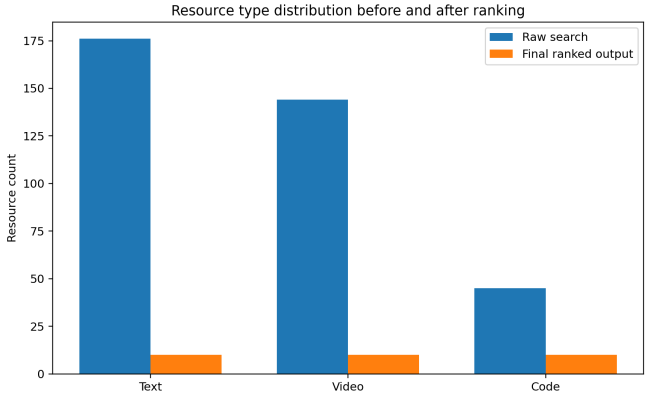

Fig. 7. Distribution of text, video, and code resources before and after recommendation compression. The final package enforces a balanced multimodal output even though raw retrieval is text-heavy.

folders, `txt/`, `video/`, and `code/`, and stores each recommended resource with source metadata, URL, and ranking score. Text resources are exported with cleaned content, while video and code items are exported as structured link summaries. In the web prototype, the same ranked resources are exposed through the interface with titles, URLs, source labels, and generated descriptions when available. This means that the implementation outcome is not merely a ranked list in memory, but a complete workflow from uploaded corpus to browsable and downloadable reinforcement materials.

One useful engineering outcome of implementation refinement was the removal of redundant summary generation in the recommendation stage. In the current version, resource summaries are generated only at final presentation time rather than during intermediate ranking, avoiding repeated work and reducing the cost of optional API-assisted summaries. This optimisation does not change the ranking logic itself, but improves the responsiveness and cleanliness of the overall pipeline.

4.2 Resource Composition and Compression Behaviour

The distribution of resource types across pipeline stages is also informative. Figure 7 shows that the raw search stage is naturally imbalanced: text resources dominate because encyclopedic and academic search results are easier to harvest in quantity than videos or code repositories. However, the final AI-Pedia recommendation set deliberately restores balance by selecting an equal number of text, video, and code items. This matters from a learning-design perspective. A student who uploads notes on transformers or neural networks rarely benefits from receiving fifteen nearly identical textual references; they are better served by a mixed package that may include a foundational explanation, an implementation repository, and a worked video explanation.

The figure therefore highlights two distinct roles of the recommendation stage. The first is *compression*: it reduces 365 candidates to 30 visible outputs across the two corpora. The second is *pedagogical balancing*: it prevents the system from simply returning the most common type of result. In the present implementation, this balancing happens by selecting top resources per modality within each corpus rather than globally ranking all resources together. This strategy is intentionally simple, but it aligns well with the educational motivation of the project, which is not just to find relevant resources but to provide complementary forms of reinforcement.

4.3 Observed Impact of Code Optimisation

Beyond paper-level design, a substantial part of the project involved refining the codebase so that the implementation better matched the intended architecture. During project inspection, several issues became clear: some evaluation scripts were not executable, the performance script did not truly measure the core pipeline, summaries were being generated redundantly, and keyword post-processing allowed near-duplicate phrases to survive into the search stage. These issues were corrected so that the system could support a more defensible evaluation. Table 3 summarises the most important changes.

These refinements are important for the dissertation because they narrow the gap between the designed system and the evaluated system. Without them, the paper would risk describing a stronger pipeline than the code actually delivered. After the changes, the codebase and the paper are much better aligned: the ranking stage performs only ranking-related work, the interface handles presentation-level summary generation, the evaluator produces the data used in the dissertation figures, and the performance script measures real local stages rather than placeholder flows.

4.4 Operational Performance

In addition to end-to-end output inspection, the project includes a small local performance harness for deterministic stages of the system. The intent is not to claim full internet-scale throughput, since external search latency depends on network conditions and third-party services, but rather to measure the intrinsic cost of the core local stages implemented by the project itself. Table 4 summarises the current measurements.

These measurements suggest that the main computational cost of the project is not the internal text processing or ranking logic, but the external retrieval stage. This is an encouraging result from an engineering perspective: the local algorithmic stages are lightweight enough to support

TABLE 3
Engineering Changes Made During Prototype Refinement

Component	Change	Practical effect
Recommender	Removed summary generation during intermediate recommendation, added lightweight domain bonus, stored final `ranking_score`.	Avoids duplicate API work, makes ranking more transparent, and improves the authority profile of the final list.
Web app	Reuses existing summaries at presentation time and only generates them lazily when absent.	Reduces repeated latency in the user-facing interface and keeps summary generation aligned with actual display needs.
Keyword extractor	Added redundancy pruning after MMR selection.	Prevents phrases such as short/long containment variants from crowding the keyword set and improves search-query usefulness.
Evaluation metrics	Reworked metrics to cover coverage, diversity, AI relevance, authority, novelty, URL validity, and modality counts.	Turns evaluation from an informal inspection task into a repeatable metric-based process suitable for inclusion in the dissertation.
Resource searcher	Added a shared HTTP session with retry-aware GET handling for repeated source queries.	Reduces repeated connection overhead and makes retrieval behaviour more stable across evaluation reruns.
Evaluation runner	Rewrote the evaluator so it can actually run end-to-end and export JSON, CSV, LaTeX tables, and chart outputs.	Makes the reported results reproducible and directly links the codebase to the paper's tables and figures.
Performance harness	Replaced placeholder timing logic with measurements of keyword extraction, ranking, and combined local pipeline stages, and exported paper-ready artefacts.	Provides genuine implementation-level performance evidence instead of superficial script execution.

TABLE 4
Local Performance of Core Deterministic Stages

Stage	Avg. time (s)	Peak memory (MB)	Notes
Keyword extraction	0.17	170.56	TF-IDF + MMR only
Ranking pipeline	0.13	170.92	CBF over cached resources
Local combined pipeline	0.16	171.64	Excludes web search latency

interactive use, while the slower part of the workflow is clearly attributable to multi-source retrieval over the web. As a result, future optimisation work should focus primarily on search orchestration, caching, and asynchronous retrieval rather than on the TF-IDF/MMR/CBF computation itself.

Figure 8 visualises the same performance data. The three bars are close in height, which indicates that keyword extraction and recommendation are of comparable computational cost at this scale. More importantly, the memory footprint remains almost flat across all three tests. This supports the design choice of relying on classical NLP and similarity-based methods for the main ranking logic: they are sufficiently lightweight to remain practical without specialised hardware.

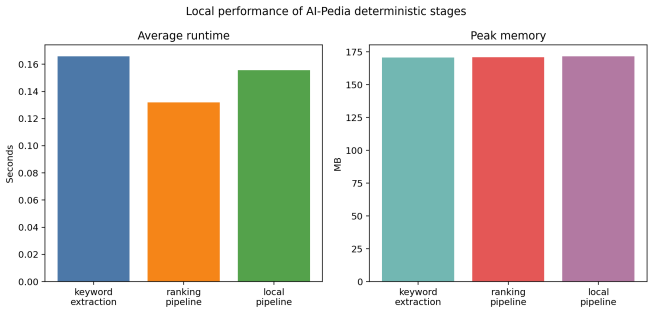

Fig. 8. Runtime and peak memory for deterministic local stages. The results indicate that the local pipeline is lightweight, and that end-to-end latency is more likely to be dominated by web retrieval than by TF-IDF/MMR/CBF computation.

5 Evaluation

This section evaluates the current implementation using comparisons that can be reproduced directly from the codebase. The aim is not yet to claim a final user-study result, but to test whether the implemented pipeline behaves sensibly across multiple coherent learner scenarios, whether the keyword extraction stage is more informative than a naive unigram baseline, and whether the ranking stage improves the quality of the final recommendation set relative to simply taking the first retrieved items.

5.1 Evaluation Design and Experimental Setup

The present evaluation uses a *within-system* design with two kinds of comparison.

First, for **RQ1**, keyword extraction is compared against a simple frequency baseline. The baseline returns the top unigrams from the corpus by word frequency after stop-word removal. The AI-Pedia condition uses the implemented TF-IDF + MMR pipeline with additional redundancy pruning. This comparison does not assume any external model and isolates the effect of the project's own topic-extraction logic.

Second, for **RQ2** and **RQ3**, the retrieved resource pool is analysed at three stages:

1) **Raw search**: all de-duplicated resources returned by multi-source retrieval;
2) **Top-K unranked**: the first five resources of each type, without CBF-based re-ranking;

3) **Top-K ranked**: the final AI-Pedia output after similarity scoring and lightweight domain-aware reranking.

This design is intentionally modest but defensible. It allows the current codebase to support concrete quantitative claims without relying on hypothetical prompts, unavailable external systems, or unimplemented annotation pipelines. The metrics used are as follows:

- **Coverage**: percentage of documents represented by at least one extracted keyword;
- **Keyword AI relevance**: percentage of extracted keywords containing AI/ML-specific terminology;
- **Keyword diversity**: lexical diversity proxy over the selected keyword set;
- **Resource AI relevance**: percentage of recommended items whose title, description, or URL contains AI/ML evidence, after normalising separators such as hyphens and underscores so repository names like `deep-learning` are not under-counted;
- **Authority score**: percentage of resources from trusted sources such as arXiv, Wikipedia, Google Scholar, or GitHub;
- **Valid URL percentage**: percentage of returned items with syntactically valid HTTP(S) URLs;
- **Noise reduction**: percentage reduction from the raw candidate set to the final displayed set.

5.2 Reproducibility Protocol

A practical strength of the current evaluation is that it is directly tied to project scripts rather than to manually assembled spreadsheet calculations. The keyword and resource metrics are produced by `test/evaluation_pipeline/evaluator.py`, while local timing measurements are produced by `test/performance/run_performance_tests.py`. The evaluator can now consume either a single corpus folder or a root containing multiple focused corpora, runs the extraction and recommendation pipeline for each scenario, writes a structured JSON report, exports CSV and LaTeX summary tables, and regenerates chart images that are reused in this dissertation. This is useful both academically and practically: it reduces the risk of reporting stale numbers and makes it easier to rerun the study after later code changes.

This reproducibility-oriented setup is especially important for a final-year project. It ensures that the dissertation can be updated consistently as the implementation changes. For example, when the recommendation logic was altered to include a domain bonus and summary reuse, the same evaluator could be rerun to observe whether the change improved authority, harmed relevance, or affected modality balance. In other words, the evaluation tooling is not an afterthought but part of the engineering contribution itself.

5.3 Keyword Extraction Results (RQ1)

Table 6 summarises the aggregate keyword comparison across the two focused corpora. The naive frequency baseline preserves full document coverage, while the TF-IDF + MMR pipeline sacrifices some literal coverage (100% to 85%) in exchange for a stronger AI-domain signal, raising keyword AI relevance from 20% to 35%. In both corpora, the implemented method produces more topic-bearing multi-word phrases than the unigram baseline.

TABLE 5
Reproducibility Assets Used in the Current Evaluation

Asset	Role in evaluation
Focused corpora	Supply two coherent ten-document AI/ML corpora for repeatable end-to-end testing.
Evaluation JSON	Stores keyword and resource metrics used for tables and plot generation.
Evaluation runner	Produces baseline-vs-pipeline metrics and exports dissertation figures.
Performance harness	Measures deterministic local stages of the implemented codebase.
Paper figures	Provide paper-ready visualisations directly generated from evaluation outputs.

TABLE 6
Keyword Extraction Aggregated Across Two Focused 10-Document Corpora

Method	Keywords	Coverage (%)	AI rel. (%)
Simple frequency baseline	10	100.00	20.00
TF-IDF + MMR	10	85.00	35.00

The selected phrases from the AI-Pedia pipeline are more descriptive than the unigram baseline. Instead of returning broad words such as *learning*, *models*, or *text*, the implemented method surfaces topic-bearing phrases such as *model evaluation*, *feature engineering*, *attention mechanisms*, and *large language models*. This is an important qualitative outcome, because the downstream search stage benefits from queries that already encode concept boundaries within each focused corpus. The lexical diversity score of the phrase-based method is slightly lower than that of the unigram baseline, and the coverage proxy also drops on the NLP corpus because a smaller set of specific phrases cannot literally touch every document. In practice, however, the phrase-based output is more usable for retrieval because it encodes topic structure rather than generic high-frequency tokens.

5.4 Resource Quality After Ranking (RQ2 and RQ3)

Table 7 reports the aggregate resource-quality comparison across both focused corpora. The raw search stage returns 365 resources across three modalities. Reducing this set to a balanced top-five-per-type recommendation list for each corpus yields an overall 91.78% reduction in candidate volume, which is substantial from a user-experience perspective because it transforms a large and noisy set into two small, manageable reinforcement packages.

Two observations are important here. First, even the raw search stage already has high AI relevance (80.55%), which indicates that the system's query construction and search-source design are directionally correct across both focused learner scenarios. Second, the ranking stage improves both *authority* and measured AI relevance in the final pooled

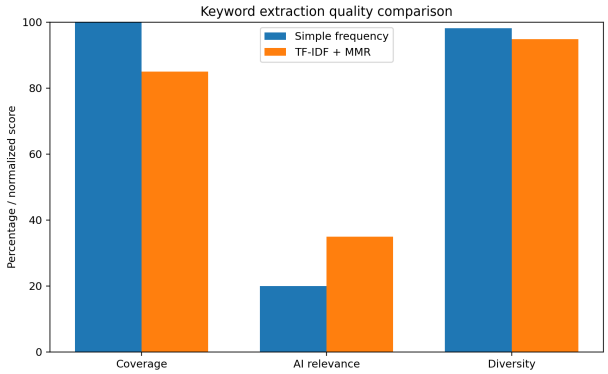

Fig. 9. Keyword extraction comparison between a simple frequency baseline and the implemented TF-IDF + MMR pipeline, aggregated across two focused corpora. The implemented method improves AI-domain focus and produces more meaningful topic phrases, although it sacrifices some literal coverage on the more specialised NLP corpus.

TABLE 7
Resource Quality Across Retrieval and Ranking Stages

Metric	Raw search	Top-K unranked	Top-K ranked
Total resources	365	30	30
AI relevance (%)	80.55	83.33	90.00
Authority score (%)	34.52	40.00	66.67
Valid URLs (%)	100.00	100.00	100.00
Noise reduction (%)	0.00	91.78	91.78
Cross-platform diversity	3	3	3

top-K set. Specifically, authority rises from 40.00% in the unranked top-K set to 66.67% in the CBF-ranked set, while AI relevance increases from 83.33% to 90.00% and valid URLs remain at 100%.

This behaviour is encouraging for an educational recommender. On the focused multi-corpus evaluation, the ranking stage again behaves as a quality-shaping mechanism rather than a superficial reshuffle. The resulting picture is cleaner: ranking improves source quality, raises measured topical relevance, and still compresses a large candidate pool into compact multimodal recommendation packages. In other words, the recommendation stage is not only filtering for similarity, but also reshaping the final list toward more trustworthy learning resources.

5.5 Interpretation and Limitations of the Current Evaluation

Taken together, the current focused-corpus evaluation supports three preliminary conclusions. For **RQ1**, the implemented keyword extraction stage is more domain-focused than a naive frequency baseline, even though its higher phrase specificity slightly reduces a literal coverage proxy. For **RQ2**, multi-source retrieval successfully produces a broad candidate pool spanning text, video, and code resources across multiple learner scenarios, and the final recommendation stage reduces that pool to balanced output sets suitable for user consumption. For **RQ3**, CBF-based ranking again appears most valuable as a *quality-shaping* mechanism, particularly by increasing the proportion of authoritative sources in the final result set.

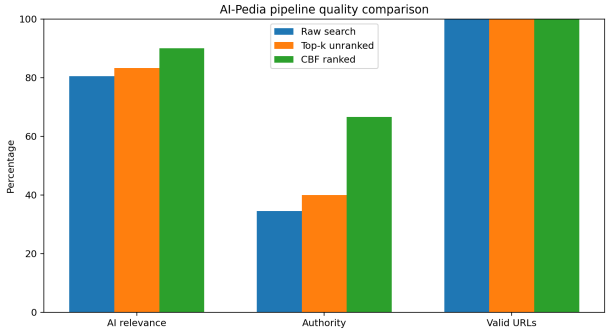

Fig. 10. Comparison of raw search, unranked top-K, and CBF-ranked top-K outputs, aggregated across two focused corpora. Ranking improves both AI relevance and source authority, and the main compression effect comes from reducing the candidate pool from 365 resources to 30.

At the same time, this evaluation remains a pilot rather than a final summative study. The setup now uses two focused corpora that each meet the system's ten-document input threshold, but the overall benchmark is still relatively small, the web sources are live and therefore volatile, and no human relevance judgements have yet been collected. In addition, metrics such as keyword diversity and AI-keyword matching are still proxy measures rather than substitutes for expert assessment. The current results should therefore be interpreted as evidence that the implementation is functioning coherently and that the main design choices are measurable, not as the final word on educational effectiveness.

The next stage of evaluation should extend this pilot in three directions. First, the benchmark should be enlarged beyond the current two focused corpora and expanded into additional topic clusters such as computer vision and reinforcement learning. Second, human annotations should be added for relevance, usefulness, and learning value. Third, a controlled user study should compare AI-Pedia with manual search or other learning-support tools. Even so, the present pilot is already valuable because it converts the paper's claims from purely architectural statements into reproducible, code-backed observations.

5.6 Threats to Validity

Several threats to validity should be acknowledged. **Internal validity** is limited by the proxy nature of some metrics: AI relevance and authority are useful indicators, but they do not directly measure whether a student learned more effectively from the returned resources. **Construct validity** is also imperfect because lexical heuristics may classify a resource as AI-related even when its pedagogical quality is weak, and may penalise useful broad resources that do not explicitly mention AI terminology in the title or description. **External validity** is constrained by the small pilot corpus and the fact that live web search results change over time. Finally, **system validity** depends partly on third-party platforms such as YouTube, GitHub, arXiv, and Google Scholar, whose interfaces and retrieval behaviour may evolve independently of the codebase.

These limitations do not invalidate the current findings, but they do frame them correctly. The present results are

best viewed as evidence that the implementation is measurable, coherent, and promising enough to justify a larger study. A stronger summative evaluation would require a larger corpus, stable annotation guidelines, repeated runs across time, and direct user feedback on usefulness and learning support.

6 Discussion

6.1 Why the Pipeline Matters for Educational Support

One of the main insights from this project is that educational recommendation for technical subjects benefits from keeping the learner's own material at the centre of the workflow. Many current AI study tools either summarise uploaded documents or answer questions directly from them. That is helpful, but it does not necessarily help a learner move outward toward high-quality reinforcement resources. AI-Pedia takes a different position: the student's own documents are used as evidence of current interest and knowledge focus, and the system then searches outward to locate complementary resources.

This distinction is important because revision and understanding often depend on triangulation. A learner may understand a concept only after reading a different explanation, seeing a worked implementation, or watching a visual walkthrough. The balanced output across text, video, and code is therefore not an incidental design feature but part of the educational argument of the system. In the pilot run, raw retrieval naturally over-produced text resources, but the final recommendation stage restored modality balance. That behaviour illustrates how recommendation logic can serve pedagogical goals rather than simply mirroring whatever type of content is easiest to retrieve in bulk.

Another useful insight is that the project benefits from classical information-retrieval techniques precisely because they are interpretable. TF-IDF, MMR, and CBF do not carry the novelty of recent large language models, but they provide understandable failure modes and measurable intermediate outputs. This matters in a dissertation context. It is much easier to justify why a keyword was chosen, why a resource received a higher score, or why a set of recommendations became more authoritative when the main ranking logic is explicit. The optional summary generation can still add convenience at presentation time, but the core recommendation behaviour remains inspectable.

6.2 Trade-offs Revealed by the Pilot Evaluation

The focused multi-corpus evaluation revealed a useful shift between compression and quality improvement. Across the two ten-document corpora, the unranked top-K resource sets already achieved strong AI relevance, but the final ranked sets improved the result further. In aggregate, the ranked output increased authority from 40.00% to 66.67% while also increasing measured AI relevance from 83.33% to 90.00%. In an educational setting, that combination is desirable: the recommendation stage is not merely filtering items down to smaller lists, but actively reshaping the output toward more trustworthy and more clearly AI-focused resources.

This observation suggests that the ranking stage should not be judged by a single metric. In fact, one contribution of the current work is to show that educational recommenders need multi-dimensional evaluation. Relevance, authority, modality balance, novelty, and accessibility all matter, and they may move in different directions after a ranking change. The current evaluation design begins to expose those relationships. Even though the corpus is small, it already shows why it is misleading to ask only whether ranking improves relevance. A more useful question is whether ranking shapes the final recommendation set in ways that better support the learner's goals.

A second trade-off concerns phrase specificity versus lexical diversity in keyword extraction. The TF-IDF + MMR stage produced better topic phrases than the simple frequency baseline, but the lexical diversity score was slightly lower. Here again, the lower score does not necessarily indicate worse behaviour. Multi-word AI concepts often share component terms, so a lexical diversity proxy can penalise semantically meaningful phrases. This result highlights the need to choose evaluation metrics that reflect downstream usefulness, not just surface-form variation.

6.3 From Prototype to Deployable System

From an engineering standpoint, the current codebase is now much closer to a deployable prototype than to a proof-of-concept script collection. The application has a working interface, modular backend components, exportable output folders, and dedicated evaluation and performance scripts. More importantly, the code was refined so that responsibilities are better separated across modules: keyword extraction performs topic discovery, search modules perform retrieval, the recommender performs ranking, and the interface handles final display and lazy summary generation. This modularity makes the system easier to test, optimise, and explain.

However, deployment-quality robustness would still require several improvements. Caching of search results would reduce repeated external latency. Stronger exception handling and retry logic would improve stability when third-party sources are temporarily unavailable. A richer configuration layer could expose ranking weights, domain priors, and source preferences without editing code. Finally, user feedback loops would allow the system to move beyond one-shot recommendation toward adaptive support. These points are not evidence of failure; rather, they show that the current prototype has reached the stage where incremental engineering work can now produce meaningful usability improvements.

Overall, the combination of code refinement and pilot evaluation strengthens the central claim of the project. AI-Pedia is not simply an idea for using AI in education; it is an implemented, inspectable, and extensible recommendation pipeline whose behaviour can already be analysed quantitatively. That makes it a solid basis for a final-year project and a credible foundation for future research or practical deployment.

7 Extended Evaluation Framework

The pilot evaluation reported in this dissertation is useful, but it should be understood as the first layer of a stronger

evaluation programme rather than as the final layer. Because the project aims to support learning rather than merely rank links, a mature evaluation strategy has to move beyond pipeline sanity checks and capture educational usefulness, recommendation quality, and system robustness together. This section therefore proposes an extended evaluation framework that can be built directly on top of the codebase developed in this project.

7.1 Phase 1: Larger-Scale Offline Benchmarking

The first next step is to scale the current pilot into a larger offline benchmark. In practical terms, this means expanding beyond the current two focused ten-document corpora and assembling additional AI/ML corpora for scenarios such as computer vision and reinforcement learning. Each corpus should represent a realistic learner scenario rather than a grab-bag of unrelated topics. For each topic cluster, multiple document sets should be created with different levels of quality and completeness, such as concise lecture notes, fragmented revision notes, textbook extracts, and mixed-source study packs.

The purpose of this phase is twofold. First, it would make it possible to test whether the keyword extraction stage remains stable across different document styles and topic granularities. Second, it would allow the ranking stage to be examined under more varied retrieval conditions, where some queries are broad, some are highly specific, and some contain noisy terminology. The current evaluator already provides much of the infrastructure needed for this phase. What is mainly missing is a broader benchmark corpus and a small amount of dataset-management tooling.

Metrics in Phase 1 should remain largely automatic so that repeated runs stay cheap. In addition to the current metrics, it would be useful to record per-topic variance, overlap between repeated runs, and source-level breakdowns of recommended materials. For example, if the system consistently over-relies on Wikipedia for broad queries or GitHub for implementation-heavy topics, that pattern would be worth identifying before a user study begins. The goal of this phase is therefore robustness analysis rather than human-impact analysis.

7.2 Phase 2: Human Annotation of Recommendation Quality

Automatic metrics are helpful, but they cannot fully answer whether the returned resources are genuinely useful to learners. A stronger study should therefore introduce human annotation. One realistic design is to sample recommendation sets from multiple corpora and ask annotators to score each returned resource on a short rubric. A four-dimension rubric would be sufficient for an undergraduate project: **topical relevance**, **authority/trustworthiness**, **pedagogical usefulness**, and **actionability**. Actionability is especially relevant for code and video resources, since a repository may be highly relevant yet difficult for a beginner to use, while a video may be accessible but too shallow to support real reinforcement.

Annotators do not necessarily need to be domain experts, although a small expert calibration step would improve quality. A practical compromise would be to use students with AI/ML coursework experience and provide a clear rubric with examples. Agreement can be checked using inter-rater statistics such as Cohen's kappa or Krippendorff's alpha if time permits. Even a small annotated sample would substantially strengthen the paper, because it would replace purely lexical proxies with judgments about usefulness.

The most important design principle here is separation between the metrics used by the system and the labels used by the evaluators. If human annotators are only asked whether a title contains AI terminology, the evaluation becomes circular. Instead, they should judge whether the resource would genuinely help a learner understand, revise, or apply the topic implied by the uploaded corpus. That makes the evaluation more faithful to the intended purpose of AI-Pedia.

A rubric of this form would allow the system to be evaluated at a level closer to its intended educational function. It would also create a practical bridge between offline benchmarking and user-centred evaluation, since many of the same dimensions, especially usefulness and difficulty fit, would later matter in live user studies.

7.3 Phase 3: User-Centred Study Design

The strongest evaluation would involve actual users interacting with the system. A manageable study for future work would compare AI-Pedia against two alternatives: manual web search and a document-grounded assistant that only answers from the uploaded corpus without recommending external resources. Participants could be given a short study packet, asked to prepare for a targeted mini-task such as explaining transformers or implementing a simple classifier, and then evaluated on both process and outcome.

Possible dependent variables include time-to-find-useful-material, self-reported confidence, perceived usefulness of recommendations, and task performance on a short quiz or implementation exercise. Even if the sample size were small, a study of this kind would add an important dimension that offline metrics cannot capture: whether the combination of text, video, and code recommendations actually helps learners navigate unfamiliar material more effectively than their baseline workflow.

It would also be valuable to collect qualitative feedback. Participants could be asked which resource types they trusted most, whether they preferred breadth or depth, and whether the balanced top-five-per-type strategy felt helpful or restrictive. These responses could inform later ranking changes. For example, some students may value authoritative text sources first and treat videos as secondary reinforcement, whereas others may learn best from code-first examples. Such feedback would point toward richer personalisation in later versions of the system.

7.4 Evaluation Hypotheses and Expected Failure Modes

An advantage of explicitly designing later evaluation phases now is that it clarifies what hypotheses the project is really making. At minimum, AI-Pedia appears to rest on four testable hypotheses. **H1**: phrase-based keyword extraction will produce better search prompts than simple frequency

TABLE 8
Proposed Human Annotation Rubric for Recommendation Quality

Dimension	Suggested scale	Interpretation
Topical relevance	1–5	How well the resource matches the concepts and skill needs implied by the uploaded corpus. Low scores indicate mismatch; high scores indicate clear topic alignment.
Authority / trustworthiness	1–5	Whether the source appears academically credible, technically reliable, and appropriate for study use.
Pedagogical usefulness	1–5	Whether the resource would plausibly help a learner understand or reinforce the topic, rather than merely mentioning it.
Actionability	1–5	Whether the learner can realistically use the resource immediately, for example by reading it, watching it, or running the code with reasonable effort.
Difficulty fit	1–5	Optional dimension capturing whether the resource is too basic, too advanced, or well matched to the likely level of the learner corpus.
Open comment	Free text	Short explanation of strengths, weaknesses, or notable mismatch. Useful for later qualitative analysis.

TABLE 9
Proposed Multi-Phase Evaluation Roadmap for AI-Pedia

Phase	Main objective	Typical inputs and outputs	Main evidence produced
Phase 1: Offline benchmark	Stress-test pipeline behaviour across multiple corpora	Multiple topic-specific corpora; automatic evaluator outputs JSON metrics and charts	Stability of keyword extraction, ranking behaviour, modality balance, source distributions
Phase 2: Human annotation	Judge recommendation quality beyond lexical proxies	Sampled recommendation lists with rubric-based ratings from annotators	Relevance, authority, usefulness, and actionability scores grounded in human judgement
Phase 3: User study	Measure learning-support value in realistic use	Participants complete search/revision tasks with AI-Pedia vs baselines	Task success, time-to-resource, confidence, satisfaction, and qualitative feedback

statistics. **H2**: multi-source retrieval will surface a broader and more useful set of reinforcement materials than any single-source strategy. **H3**: ranking will improve the overall usefulness of the final set even when it does not maximise every lexical relevance proxy. **H4**: learners will benefit from multimodal reinforcement compared with workflows based only on document upload or manual search.

Making these hypotheses explicit is useful because it also reveals likely failure modes. For H1, the system may fail when uploaded documents are extremely short or noisy. For H2, the main risk is that live web retrieval changes over time and introduces volatility. For H3, ranking may over-prioritise authority and under-prioritise niche but valuable resources. For H4, the system may work well for exploratory learning but less well for users who already know exactly which format they prefer. A robust evaluation programme should therefore measure not only average performance but also where and why the system breaks down.

7.5 Implications for the Current Dissertation

Even though all three phases are not yet complete, laying out this framework strengthens the dissertation in two ways. First, it shows that the current pilot evaluation is part of a coherent research trajectory rather than a disconnected set of metrics. Second, it demonstrates that the code and data artefacts developed in the project are already organised well enough to support more ambitious studies. The evaluator generates reusable outputs, the figures can be regenerated from JSON, and the modular codebase makes ablation and extension feasible.

In this sense, the project should be judged not only by the current numerical scores, but also by how well it establishes the methodological foundation for a stronger next-stage study. A final-year project cannot usually deliver every layer of an educational recommender evaluation, but it can deliver a defensible prototype, a transparent measurement framework, and a clear path to deeper validation. AI-Pedia now does all three.

7.6 Statistical Reporting Strategy for Future Runs

A final design issue worth making explicit is how future runs should be reported once the corpus and annotation budget grow. For offline benchmarking, single-score reporting is not enough. Each experiment should ideally be rerun over multiple corpora and, where retrieval volatility matters, at multiple times. Results should then be summarised using means, standard deviations, and per-topic breakdowns rather than only global averages. This would make it easier to identify whether one component of the pipeline is consistently strong or whether performance is being driven by one especially easy topic cluster.

For pairwise comparisons such as baseline versus TF-IDF + MMR, or unranked top-K versus CBF-ranked top-K, the most useful reporting style would combine descriptive and inferential summaries. Descriptive reporting should include absolute scores, percentage deltas, and plots showing variability across corpora. Inferential reporting could then use paired tests if the dataset size becomes large enough, for example paired t-tests or non-parametric alternatives across matched corpus conditions. In a project of this scale,

significance testing is less important than transparent effect reporting, but even simple paired comparisons would add rigour.

Human-annotation results should likewise be reported with more structure than a single average score. If annotators rate relevance, usefulness, authority, and actionability separately, the paper can show where the system is strong and where it remains weak. For instance, a future version of AI-Pedia might perform well on authority and actionability while remaining only moderate on pedagogical usefulness for beginners. Reporting the four dimensions separately would make such patterns visible and would guide future ranking-weight changes more effectively than a collapsed composite score.

Finally, qualitative feedback should be treated as data rather than anecdote. If participants repeatedly say that code repositories are valuable only when accompanied by explanatory text, or that videos are helpful for orientation but not for deep revision, those patterns can inform subsequent system design. In short, the next-stage evaluation should combine automatic metrics, human ratings, and user narratives in a way that reflects the genuinely multi-dimensional nature of educational recommendation.

8 Conclusion

This paper presented AI-Pedia, an AI-powered multimedia resource recommendation system for machine learning education. The system addresses a practical gap in existing study workflows: learners often have a private corpus of notes and readings, but still need support in locating high-value external reinforcement materials across text, video, and code platforms.

AI-Pedia's core contribution is a learner-driven reinforcement pipeline that transforms a student's private AI/ML materials into curated external resource recommendations. The system combines established information retrieval and recommender-system methods (TF-IDF, MMR, and CBF) with multi-source retrieval from Wikipedia, Google Scholar, arXiv, YouTube, and GitHub. The current focused-corpus evaluation shows that the implementation is operational and measurable across more than one learner scenario: the keyword extraction stage improves AI-domain focus over a naive frequency baseline (35% vs. 20% keyword AI relevance), the retrieval stage generates a broad multimodal candidate pool, and the final ranked recommendation set reduces raw candidates by 91.78% while increasing authority from 40.00% to 66.67% and AI relevance from 83.33% to 90.00% relative to an unranked top-K set.

These pilot results do not yet constitute a final user-study claim, but they do support the central design rationale of the project. In particular, they show that the modular pipeline is not merely architecturally plausible; it produces reproducible outputs, exposes measurable tradeoffs, and is sufficiently interpretable to support further evaluation and refinement.

Key lessons learned during this project include:

- **Modularity enables evaluation**: Designing the system as a modular pipeline makes it possible to evaluate keyword extraction, retrieval, and ranking separately rather than treating the recommender as a black box.
- **Query quality matters**: Phrase-based keyword extraction produces more useful search queries than a unigram-only baseline, even on a small pilot corpus.
- **Ranking shapes quality**: The most visible benefit of the current ranking stage is improved source authority, showing that ranking can steer the final output toward more trustworthy reinforcement materials.
- **Engineering details affect usability**: Removing duplicate summary generation reduced unnecessary work and made the overall pipeline cleaner and easier to evaluate.

8.1 Limitations

We acknowledge several limitations of this work:

- **Evaluation scope**: The present evaluation is a pilot on two focused ten-document corpora rather than a full-scale user study, so the reported numbers should be treated as formative evidence.
- **Domain specificity**: The system was designed and evaluated for AI/ML education; adaptation to other domains would require curating domain-specific keyword lists and potentially adjusting filtering rules.
- **Resource volatility**: External resources, especially YouTube videos and GitHub repositories, may change or become unavailable over time, affecting recommendation stability.
- **External latency**: The slowest part of the full workflow is live web retrieval rather than local ranking, so responsiveness depends partly on third-party services and network conditions.

8.2 Future Work

Several directions for future work emerge from this project:

1) **Expanded evaluation**: Conduct a longitudinal study tracking learning outcomes over weeks or months, comparing students who use AI-Pedia with those who rely on manual search or generic assistants.
2) **Domain adaptation**: Extend the system to support additional domains (e.g., mathematics, physics, programming) by curating domain-specific keyword lists and training custom filtering models.
3) **Enhanced personalization**: Incorporate learner modeling to adapt recommendations based on prior knowledge, learning goals, and preferred learning styles.
4) **Interactive refinement**: Allow users to provide feedback on recommendations (e.g., "more like this" or "less like this") and use this feedback to refine future suggestions.
5) **Resource freshness monitoring**: Implement periodic re-validation of recommended resources to detect and replace broken links or outdated content.
6) **Integration with learning platforms**: Explore integration with existing learning management systems (e.g., Canvas, Moodle) or note-taking tools (e.g.,

TABLE 10
Summary of Key Findings from the Current Prototype

Area	Main finding
Keyword extraction	TF-IDF + MMR improved keyword AI relevance from 20.00% to 35.00%, while sacrificing some literal coverage in exchange for more topic-specific phrases.
Resource retrieval	Multi-source search returned 365 candidates across text, video, and code resources over two focused learner scenarios.
Recommendation stage	Final output compressed 365 candidates to 30 balanced recommendations with 91.78% noise reduction.
Ranking effect	CBF-based ranking increased authority from 40.00% to 66.67% and AI relevance from 83.33% to 90.00%.
Local performance	Deterministic local stages remained lightweight, with sub-0.2 second average runtime and about 171 MB peak memory.
Engineering outcome	Evaluation and performance scripts now produce reproducible outputs suitable for direct inclusion in the dissertation.

Notion, Obsidian) to embed AI-Pedia into existing student workflows.

In conclusion, AI-Pedia demonstrates that a reinforcement-oriented recommendation system can meaningfully support AI/ML learners by connecting their private study materials to high-value external resources. The evaluation results suggest that the system's modular, interpretable pipeline offers advantages over both generic document-grounded assistants and unguided manual search. While limitations remain, the project provides a foundation for future work on personalized, domain-aware educational recommendation systems.

REFERENCES

[1] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] J. Carbonell and J. Goldstein, "The use of MMR, diversity-based reranking for reordering documents and producing summaries," in *Proc. SIGIR*, 1998, pp. 335–336.

[3] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge Univ. Press, 2008.

[4] M. J. Pazzani and D. Billsus, "Content-based recommendation systems," in *The Adaptive Web*, P. Brusilovsky, A. Kobsa, and W. Nejdl, Eds. Berlin, Germany: Springer, 2007, pp. 325–341.

[5] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, Eds., *Recommender Systems Handbook*. Boston, MA, USA: Springer, 2011.

[6] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.

[7] R. E. Mayer, *Multimedia Learning*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[8] S. N. Kumar *et al.*, "Multimedia-based learning tools and its scope, applications for virtual learning environment," in *Computational Intelligence in Digital Pedagogy*. Singapore: Springer, 2020, pp. 47–63.

[9] A. Vaswani *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[10] R. Bommasani *et al.*, "On the opportunities and risks of foundation models," Stanford CRFM, Tech. Rep., 2021.

[11] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[12] D. B. Rajesh and A. Kumar, "Collaborative filtering models: An experimental and detailed comparative study," *Scientific Reports*, vol. 15, Art. no. 31667, 2025. [Online]. Available: https://doi.org/10.1038/s41598-025-15096-4

[13] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 6, pp. 734–749, Jun. 2005.

[14] M. Shanmugamathan and S. Almutairi, "A user-based video recommendation approach using CAC filtering, PCA with LDOS-CoMoDa," *The Journal of Supercomputing*, vol. 78, no. 15, pp. 1–15, May 2022.